

AI Interaction Progress

Checkpoint 3



Table of Contents

Table of Contents....2
AI Interaction Summary....3
iOS Development Environment on the MacBook Displaying Encoder Information Key Prompts + Problems....4
Motor Control Fully App Based Additions to Code Key Prompts + Problems....5
Encoder Functionality Key Prompts + Problems....6
Launch on Startup....7
Test A : RPM Sensing Through the App....8
Test B: Open Loop Missed Steps....11
Updated Software Documentation....13
GitHub Site Creation....14

AI Interaction Summary

AI Interaction	AI Contribution (all ChatGPT except where stated)	Confirmation
Magnetic Encoder Functionality (C Code)	Generated motor_cp3.c and motor_cp3.h which contained the functionality for the magnetic encoder to work with the app	Confirmed the magnetic encoder angle reading matched physical protractor reading, and confirmed magnetic encoder RPM was within 5% of commanded RPm
Motor Control Fully App Based Additions to Code (ARM assembly and C code)	Generated updated motor_cp3.c, motor_cp3.h, and parmco_ble_cp3.c to include appropriate controls and limits based on the app	App build and deployment to iPhone 17 Pro and bluetooth connection to RP4, verified all required functions were present, confirmed button responsiveness and functionality
Calibrate/Set to Zero app control + Angle Target Code Additions	Updated swift files to include calibration widget + indicator if it has been calibrated or not	Button responsiveness, angle target matching physical protractor, set to 0 matched physical protractor 0
Creating RPM Reporting in the App	Updated swift files to include a widget showing IR RPM, Encoder RPM, Encoder Angle, and calibration status	Ensuring IR & Encoder RPM was within 5% of commanded RPM after settling, encoder angle matched the physical protractor based on a reference blade
Test B: Open Loop Missed Steps - option to overdrive RPM	Updates swift files (ContentView.swift) to include toggle to switch into overdrive mode, allowing RPMs to reach beyond 100 RPM	Checked that RPM could be set beyond 100 when toggled on, restricted when toggled off
Logging System for Writeup Data	Update ContentView.swift to include a button to start and stop logging, save to folder called “logs”	Confirm the CSV files contained expected data and all required metrics, all files present in log folder
Updated Software Documentation	Based on previous documentation, feedback, and new progress, generated updates & detailed documentation	Confirmed sufficient detail, correct information, and that no contradictions were present
GitHub Site Creation	Generating directions for how to activate the pages functionality on GitHub and html file to link and display all the PDFs for the various docs.	Confirmed by testing and launching a live website that correctly linked the information required and it is accessible to the public.

iOS Development Environment on the MacBook Displaying Encoder Information Key Prompts + Problems

Prompt: [ChatGPT Plus] Now I am moving on to checkpoint 3. NOTE: do not implement the launch on startup until I ask you to. I want to begin by implementing the full app control of the motor (except full feedback mode). I have provided the detailed checkpoint 3 requirements as well as the previous code files. Please request the class library if required and any additional documentation (such as the full project description). give me the files in text.
Result: Has features required for the final checkpoint, and removed existing controls from checkpoint 2, view files at BluetoothManager_BadUI.swift and ContentView_BadUI.swift .
Problems: The UI is broken, buttons are unresponsive, and formatting is irregular and hard to read. It did not preserve existing controls from checkpoint 2, and added additional controls only required for the final checkpoint.

Prompt: [ChatGPT Plus] Let's focus on the app first. use my original app from cp2 as a starting point. Essentially, all the keyboard command functionality from checkpoint 2 now needs to be controlled by the app in checkpoint 3 (except full feedback mode). The default controls shown should be 20 rpm, manual mode, clockwise, with the motor stopped. The full requirements of the functionality of the app interface is in the project screenshot attached: I provided the screenshot with the controls the app should support as well as the app code files you should start from. We will only be focusing on building this swift UI first. I provided the test A and B information as well for reference, and the entire project instructions to reference ONLY FOR BUILDING OUT THE APP UI FOR THIS CHECKPOINT 3! [Attached all three swift files, project instructions, screenshots with specific checkpoint 3 instructions]
Result: Generated updated swift files ContentView.swift and BluetoothManager.swift. The UI now contains the following features: bluetooth connection, control mode, desired speed, direction, motor state, bluetooth messages, live sensor status, position controls, logging and tests, and additional controls. View final UI files at AppSwiftFiles .
Problems: No problems, required controls and app UI structure has all the required features.

Motor Control Fully App Based Additions to Code Key Prompts + Problems

Prompt: [ChatGPT Plus] Now, let's implement the functionality on the RP4/C side, starting for checkpoint 3. It should allow the motor to be controlled by the app. keep in mind when the motor is set to run, it should keep running until the user presses stop in the app. The checkpoint 3 guidelines, full project guidelines, and code from cp2 are provided as a reference. At this point, make sure to just write code so all the controls in the app are able to physically control the motor, do not worry about the magnetic encoder at this time.
Result: Generated updated parmco_ble_cp3.c which supports the bluetooth server, executes motor commands,reports live status. New files motor_cp3.c and motor_cp3.h for motor control, continuous running, IR RPM, and logging. View files here: motorDriver_frozen.s, motor_cp3_frozen.c, motor_cp3_frozen.h, parmco_ble_cp3_frozen.c.
Problems: After connecting with the app, the app can start the motor but after starting the motor pressing anything in the app will not cause the motor functionality to change. Essentially, functionality is frozen after starting the motor.

Prompt: [ChatGPT Plus] Here is the functionality error now: after pressing start motor, the motor behavior doesn't change in response to any of the buttons being pressed. The IR reading is not updating either. Essentially the motor functionality is unable to be changed by the app after pressing start motor. Investigate this issue and provide the corrected C code files.
Result: Generated the corrected motor_cp3.c, parmco_ble_cp3.c, and motor_cp3.h files addressing bluetooth responsiveness, IR RPM, and continuous rotation issues. View files here: motor_cp3_withoutencoder.c, motor_cp3_withoutencoder.h, parmco_ble_cp3_withoutencoder.c.
Problems: None, works as expected.

Encoder Functionality Key Prompts + Problems

Prompt: [ChatGPT Plus] Now let's work on the magnetic encoder functionality with tests A and B, and zero calibration. The only things interacted with are the SDA (GPIO 2) and SCL (GPIO 3) pins from the encoder. I provided the project instructions, current C files, and checkpoint 3 instructions as a reference point.
Result: Generated updated motor_cp3.c, parmco_ble_cp3.c, and motor_cp3.h files preserving previous functionality but adding the magnetic encoder functionality including magnetic encoder readings, encoder angle, encoder RPM, calibrate zero, set to zero, angle target, test A, test B, and logging. View here: motor_cp3_encErr.c, motor_cp3_encErr.h, parmco_ble_cp3_encErr.c.
Problems: The RP4 program outputs error messages related to the encoder readings.

Summary of setup issues		
Issue	Details	AI Result
ERR:ENCODER_NOT_READY	Means the RP4 is failing to recognize a reading from the sensor	Provided commands to run in the terminal for troubleshooting, check terminal output, verification of encoder connection
Encoder is not producing a valid reading	Provided terminal output to AI	Reboot sudo, check if sensor is detected, magnet detection status,
Issue with tracking the motor shaft	A significant physical degree change is only read as 0.7 degree change	Provided debugging code, and then identified the issue to be the register-read transaction in E4235 class library code, updated code files as necessary (final code)
The angle command does not work as expected	When commanded to go to 90 degrees, it went to either 70 or 80 degrees	Tried a few different code solutions, linux commands, ultimately recommended changing the blade as it moves independently of the shaft - changing the blade resolved the issue

Launch on Startup

Prompt: [ChatGPT Plus] Final step: Walk me through how to setup launch on startup for the parmco_cp3 executable.

Result:
sudo nano /etc/systemd/system/parmco.service
—----------

```
[Unit]
Description=PARMCO Raspberry Pi Motor Control
After=bluetooth.service dbus.service
Wants=bluetooth.service

[Service]
Type=simple
User=root
WorkingDirectory=/home/group1/Group-1/Final_Project/CP3
ExecStart=/home/group1/Group-1/Final_Project/CP3/parmco_cp3

Restart=always
RestartSec=5

KillSignal=SIGTERM
TimeoutStopSec=20

StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

—----------

```
sudo systemctl daemon-reload
sudo systemctl enable parmco.service
sudo systemctl start parmco.service
sudo systemctl status parmco.service
sudo journalctl -u parmco.service -f
sudo reboot
```

Whenever C code is updated, run: sudo systemctl restart parmco.service

Problems: None

Test A : RPM Sensing Through the App

Prompt: [Copilot Pro With Claude Integration]
Built an Agent that would calculated the required data for Test A
Agent Directions:

https://m365.cloud.microsoft/chat/?titleId=T_4958ef29-fb76-474c-b3d2-53f1437d9987&source=embedded-builder

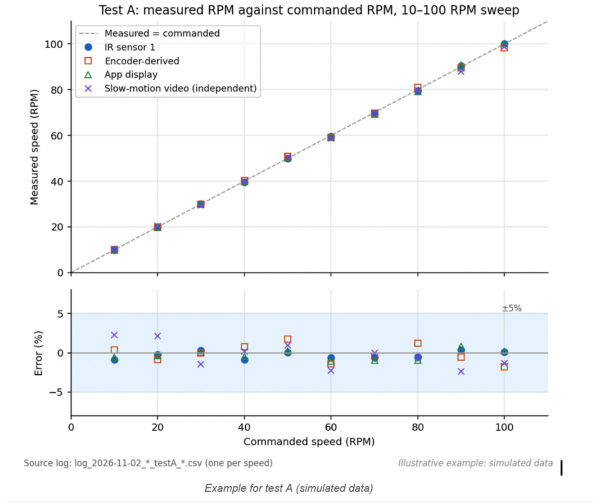
Build Directions:
Purpose
Analyze motor-speed log files and compare measured RPM values with commanded RPM for each tested speed.

General Guidelines
Use the uploaded log as the source of truth.
Identify the commanded-speed, ir_rpm, enc_rpm, and app_rpm fields even when column names vary slightly.
Use the code interpreter for calculations and file handling.
Preserve full precision during calculations and round only the final displayed results.
Present assumptions clearly and keep explanations concise and technical.
Step-by-Step Instructions
Inspect the log. Identify speed groups, measurement columns, timestamps, pulse counts, encoder counts, sampling intervals, and calibration constants.
Validate inputs. Confirm units and exclude malformed or missing rows from each affected calculation. Report how many rows were excluded.
Calculate averages. For each commanded speed, calculate the arithmetic mean of ir_rpm, enc_rpm, and app_rpm over the corresponding log records.
Calculate independent RPM. Prefer a raw derivation that does not reuse the three reported RPM fields, such as revolutions or pulse counts divided by elapsed time. Apply pulses-per-revolution, gear ratio, and sampling interval when supplied. If the log lacks enough information, state exactly which value or calibration constant is needed instead of inventing it.
Calculate percent error. For each RPM estimate, use absolute(measured RPM - commanded RPM) / absolute(commanded RPM) × 100. Mark percent error as undefined when commanded RPM is zero.
Report results. Return one row per commanded speed with commanded RPM, average IR RPM, average encoder RPM, average application RPM, independent RPM, and the percent error for each estimate.
Summarize findings. Identify the most accurate method at each speed and note systematic bias, instability, or outliers visible in the log.
Error Handling
Ask for the log file if none is available.
If column meanings are ambiguous, show the inferred mapping and ask only for the minimum clarification needed.
Do not silently substitute missing values or combine different commanded-speed groups.

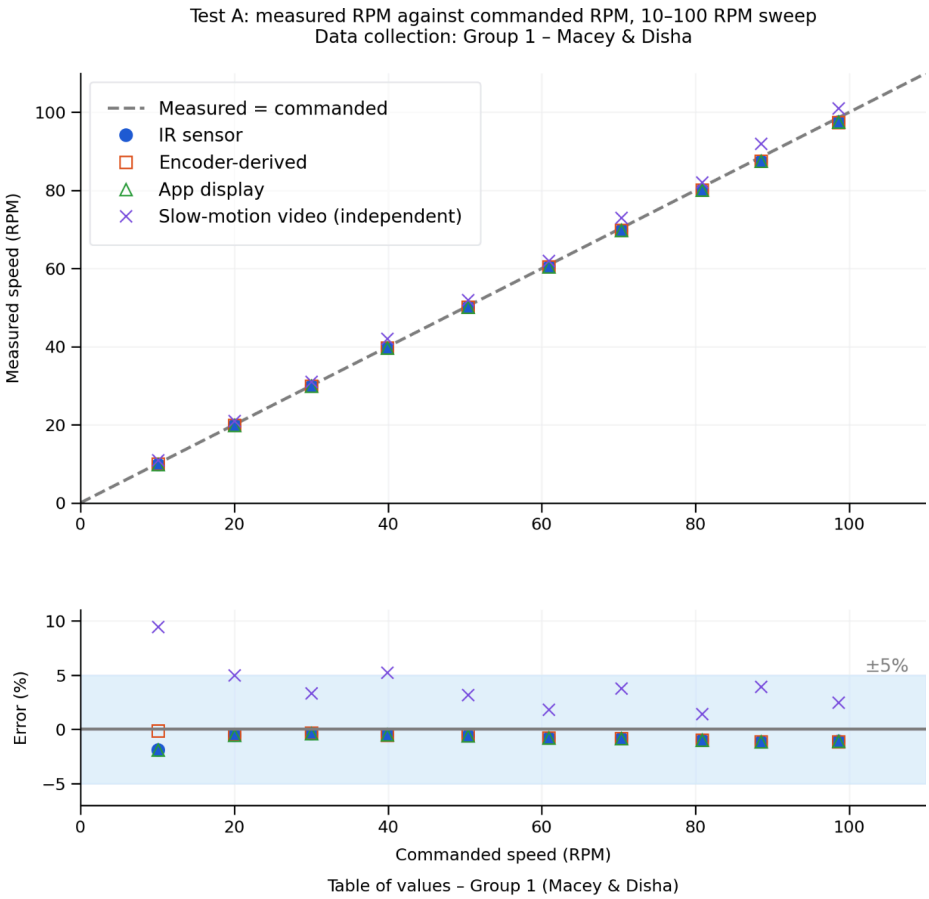
Result: Created the agent that calculated all of the data in the table found Section Test A

Problems: None.

Prompt: [Copilot Pro With Claude Integration] Make this graph with my measured data: how: measured RPM against commanded RPM for all four sources with a y = x reference line, and a percent-error panel with a ±5% band. Take from the already provided blade count data for trials 10-100.



*Attached table of values from tests, and example test A graph provided in directions appendix



Result:

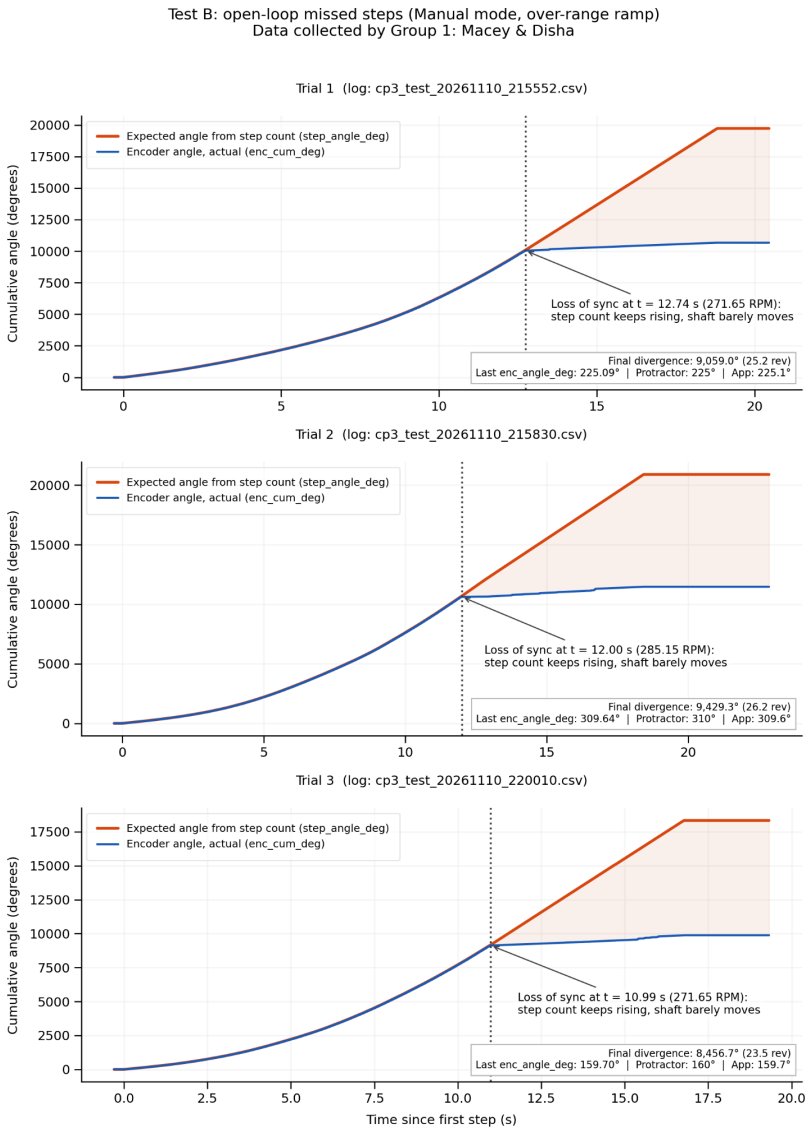
Problems: Not a problem, but wanted to add a value table in the future.

Test B: Open Loop Missed Steps

Prompt: [Copilot Pro With Claude Integration] This is the file for the Trial one, Calculate: the time the two angles first differ by more than one step, and the total divergence at the end. Check that the last enc_angle_deg matches your protractor reading. This is the file for the Trial two, Calculate: the time the two angles first differ by more than one step, and the total divergence at the end. Check that the last enc_angle_deg matches your protractor reading. This is the file for the trial three, Calculate: the time the two angles first differ by more than one step, and the total divergence at the end. Check that the last enc_angle_deg matches your protractor reading.
Result: Table sited in Part B for the three trial runs derived from the collected data.
Problems: None.

Prompt: [Copilot Pro With Claude Integration] Calculate: the time the two angles first differ by more than one step, and the total divergence at the end. Check that the last enc_angle_deg matches your protractor reading.Trial 1: actual 225 degrees, app 225.1 deg Trial 2: actual 310, app 309.6 Trial 3: actual 160, app 159.7
Result: All graphs shown in Part B of the writeup with the proper calculated values.
Problems: None

Prompt: [Gemini Pro] For the three trials for test B, Show: step_angle_deg and enc_cum_deg against time on one plot, with the loss of sync marked. make this graph format with new the 3 trial data, and add data collected by Group 1: Macey and Disha ***Attached provided Table B format from the directions**



Result: Half-step mode (0.9°/step). Encoder zeroed at first step; slip time corrected for −20 ms encoder update delay.

Problems: None, graphs looked great.

Updated Software Documentation

Prompt: [ChatGPT Plus] I need to create updated software documentation based on this week's progress with library attribution, how each library works and why it is needed. Code flags must also be present. This document should cover documentation and make updates for motor_cp3.c, motor_cp3.h, parmco_ble_cp3.c, motorDriver.s and the PARMCO app files (PARMCOapp.swift, BluetoothManager.swift, ContentView.swift), all of which are attached. Generate a pdf with these details, ensuring documentation is consistent with the provided code behavior and formatting of the previous document (attached). The full project instructions are also attached for reference - only refer to sections pertaining to the ai software documentation for checkpoint 3. Request additional documents/information required, but do not hallucinate results and remove sections that are no longer relevant. Generate the new pdf.
Result: Generated a new draft of the AI software documentation: PARMCO_CP3_Software_Documentation_Version1.pdf
Problems: There are issues with comparisons between CP2 and CP3, unprofessional language, and other small issues (specified in the next prompt).

Prompt: [ChatGPT Plus] A few changes - remove the scope boundary and any text comparing cp2 and cp3 (remove key changes etc) , and also don't call this one the cp3 version (just refer to it as the project, forget there was a previous version) in the document, remove it. Remove the course-rule exception to solve. Also, the most recent swift file lets it go up to 400 rpm and remove 10.1 additional information to finalize release. In 11.1 remove the column that says "attached snapshot filename". In general, language needs to be more professional. Make these updates and provide the fully corrected pdf.
Result: Final generated AI software documentation: PARMCO_CP3_Software_Documentation_Final.pdf
Problems: None, issues fixed in PDF.

GitHub Site Creation

Prompt: [Copilot Pro With Claude Integration] The site is published but only shows the read me, how do I let it show my folders of PDFs with documentation on the pages
Result: Your site only shows the README because GitHub Pages doesn't list folders. If there's no `index.html`, it turns `README.md` into the homepage and nothing else. To show your PDFs, add an `index.html` at the root of the repo that links to each file.
Problems: No problems, the information was valid and helpful.

Prompt: [Copilot Pro With Claude Integration] here are the folders with the pdfs, help me make it so that the website shows them in the nicely formatted way you have it in STEP 2 **(Attached were screenshots of our document structure with pdf names and file paths)**
Result: Generated the HTML file that correctly linked to the schematics, diagrams, and documents we had, documenting the complete process of the project up until this point. This file on the live site was generated: index.html *Note: the Ai Interaction Reflection Section was human written by Macey Glad
Problems: No problems, generated and formatted the required files perfectly.

Prompt: [Copilot Pro] how do I make a live github site using gihub pages
Result: The usual way to do this is **GitHub Pages**. It's free, and it turns a repository into a live site at `https://<your-username>.github.io/<repo-name>/`. To get full marks on this rubric line, the site needs to be **live, with structure and early content that describes the project**. **The rest of the prompt outlined the exact steps it would take to activate and create a live site using GitHub pages, and it was easy to manually follow on my personal github account.**
Problems: None